CO3-AT3-Assignment

← Back

QUESTIONS

Q1
Banking Transaction System – Exception Handling
10 marks

Q2
Hospital Patient Registration – User-Defined Exception
10 marks

Q3
E-Commerce Order Processing – Built-in Exceptions
10 marks

Q4
Hospital Emergency System – Thread Priority
10 marks

Q1 Banking Transaction System – Exception Handling

10 marks

Develop a Java program for an ATM withdrawal system. The program accepts the account balance and withdrawal amount.

The program must:

- Accept the current account balance and withdrawal amount.
- Throw a user-defined InsufficientBalanceException when the withdrawal amount exceeds the balance.
- Handle withdrawal amounts that are zero or negative.
- Handle invalid numeric input using a suitable built-in exception.
- Display a meaningful transaction result.
- Use try-catch-finally so that the transaction completion message is always displayed.

Sample Test Cases:

Input / Scenario	Expected Result	Purpose
10000 3000		Normal case
10000 12000		Boundary / exception case
10000 -500		Invalid input
abc 3000		Invalid input

Secure Exit

SIMATS Online Programming Lab

JAVA PROGRAMMING

K.Dhilip
192365013RD

Your Code

```java
import java.util.*;
public class Main{
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);
        try{
            int balance=sc.nextInt();
            int amount=sc.nextInt();
            if(amount<=0){
                throw new IllegalArgumentException("Withdrawal amount n
            }
            if(amount>balance){
                throw new InsufficientBalanceException("Insufficient ba
            }
            balance=balance-amount;
            System.out.println("Withdrawal successful.");
            System.out.println("Remaining Balance = "+balance);
        }
        catch(InsufficientBalanceException e){
            System.out.println(e.getMessage());
        }
        catch(IllegalArgumentException e){
            System.out.println(e.getMessage());
        }
        catch(Exception e){
            System.out.println("Invalid input.");
        }
        finally{
            System.out.println("Transaction completed.");
        }
    }
}
class InsufficientBalanceException extends Exception{
    InsufficientBalanceException(String message){
        super(message);
```

Input

```
abc 3000
```

Output

```
Invalid input.
Transaction completed.
```

Exit

SIMATS Online Programming Lab

JAVA PROGRAMMING

K.Dhilip
192365013RD

← Back

Q2 Hospital Patient Registration – User-Defined Exception

10 marks

Develop a Java program for a hospital patient registration system. The program accepts the patient's name, age, and body temperature. Create suitable user-defined exception classes for invalid age and invalid temperature. Validate the input before registration. Use try-catch-finally and display a registration completion message from finally.

Sample Test Cases:

Input / Scenario Expected Result Purpose

Arun, 35, 98.6 Patient registered successfully Normal case

Meena, 12, 98.4 InvalidAgeException Boundary / exception case

Ravi, 45, 105.2 InvalidTemperatureException Invalid input

Kumar, -2, 98.1 InvalidAgeException Invalid input

Exit

SIMATS Online Programming Lab

JAVA PROGRAMMING

K.Dhilip
192365013RD

Your Code

```java
import java.util.*;
public class Main{
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);
        try{
            String name=sc.next();
            int age=sc.nextInt();
            double temperature=sc.nextDouble();
            if(age<18||age>100){
                throw new InvalidAgeException("Invalid age");
            }
            if(temperature<95.0||temperature>104.0){
                throw new InvalidTemperatureException("Invalid temperat
            }
            System.out.println("Patient registered successfully");
        }
        catch(InvalidAgeException e){
            System.out.println("InvalidAgeException");
        }
        catch(InvalidTemperatureException e){
            System.out.println("InvalidTemperatureException");
        }
        finally{
            System.out.println("Registration completed.");
        }
    }
}
class InvalidAgeException extends Exception{
    InvalidAgeException(String message){
        super(message);
    }
}
class InvalidTemperatureException extends Exception{
    InvalidTemperatureException(String message){
```

Input

```
Kumar -2 98.1
```

Output

```
InvalidAgeException
Registration completed.
```

← Back

Q3 E-Commerce Order Processing – Built-in Exceptions

10 marks

Develop a Java program for an online shopping application. Store product names and prices in arrays and accept a product index and quantity from the customer.

The program must:

- Calculate total cost as quantity × price.
- Handle invalid product indices.
- Handle invalid numeric input.
- Reject zero or negative quantities.
- Use appropriate built-in exception classes.
- Continue execution after handling an invalid transaction.

Sample Test Cases:

Input / Scenario Expected Result Purpose

1, 2 Valid product; total cost calculated Normal case

5, 2 ArrayIndexOutOfBoundsException handled Boundary / exception case

2, -1 Invalid quantity Invalid input

abc, 2 InputMismatchException handled Invalid input

Your Code

```java
import java.util.*;
public class Main{
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);
        String[] products={"Laptop","Phone","Headphones"};
        double[] prices={50000,20000,3000};
        try{
            int index=sc.nextInt();
            int quantity=sc.nextInt();
            if(quantity<=0){
                throw new IllegalArgumentException("Invalid quantity");
            }
            double total=prices[index]*quantity;
            System.out.println("Total Cost = "+total);
        }
        catch(ArrayIndexOutOfBoundsException e){
            System.out.println("ArrayIndexOutOfBoundsException handled"
        }
        catch(InputMismatchException e){
            System.out.println("InputMismatchException handled");
        }
        catch(IllegalArgumentException e){
            System.out.println("Invalid quantity");
        }
    }
}
```

Submitted

Input

```
abc 2
```

Output

```
InputMismatchException handled
```

Q4 Hospital Emergency System – Thread Priority

10 marks

A hospital application creates threads for Emergency Patient Processing, General Patient Processing, and Billing Processing.

Develop a Java program that assigns different priorities to these threads. Run the program several times and analyze whether the emergency thread is guaranteed to execute first.

Then modify the design if the hospital requires a strict execution sequence rather than merely expressing scheduling preference.

Your answer must contain Java code and an execution analysis based on the actual thread behavior.

Sample Test Cases:

Input / Scenario Expected Result Analysis Focus

Emergency, General, Billing All threads execute; priority does not guarantee a fixed output order Normal execution

Emergency workload > General workload Emergency thread receives a higher priority but exact ordering is not guaranteed Concurrency / design

Strict emergency-before-billing requirement Use explicit coordination rather than relying only on priority Exception / boundary

Your Code

```java
import java.util.*;
public class Main{
    public static void main(String[] args){
        EmergencyThread emergency=new EmergencyThread();
        GeneralThread general=new GeneralThread();
        BillingThread billing=new BillingThread();
        emergency.setPriority(Thread.MAX_PRIORITY);
        general.setPriority(Thread.NORM_PRIORITY);
        billing.setPriority(Thread.MIN_PRIORITY);
        emergency.start();
        general.start();
        billing.start();
    }
}
class EmergencyThread extends Thread{
    public void run(){
        System.out.println("Emergency Patient Processing");
    }
}
class GeneralThread extends Thread{
    public void run(){
        System.out.println("General Patient Processing");
    }
}
class BillingThread extends Thread{
    public void run(){
        System.out.println("Billing Processing");
    }
}
```

Input

stdin input

Output

```
Emergency Patient Processing
Billing Processing
General Patient Processing
```

Submitted

Q5 Online Food Delivery – Multiple Threads and Execution Order 10 marks

An online food-delivery application uses three threads: Order Processing, Payment Processing, and Delivery Assignment.

Develop a Java program in which these activities are represented by separate threads. Analyze the problem if all three threads are started independently.

Modify the program so that:

- Order processing completes before payment processing.
- Payment processing completes before delivery assignment.
- The program uses appropriate Java thread-management techniques.
- The output clearly shows the required execution order.

Explain the difference between simply starting all threads and explicitly controlling their execution order.

Sample Test Cases:

Input / Scenario Expected Result Analysis Focus

Order=Pizza; Payment=UPI; Delivery=Agent A Order → Payment → Delivery Normal execution

Order=Burger; Payment=Card; Delivery=Agent B Order → Payment → Delivery Concurrency / design

Payment=Failed Delivery assignment should not proceed Exception / boundary

Your Code

```java
import java.util.*;
public class Main{
    public static void main(String[] args)throws InterruptedException{
        Scanner sc=new Scanner(System.in);
        String input=sc.nextLine();
        String[] data=input.split(";");
        String order=data[0].split("=")[1].trim();
        String payment=data[1].split("=")[1].trim();
        String delivery=data[2].split("=")[1].trim();
        OrderThread t1=new OrderThread(order);
        PaymentThread t2=new PaymentThread(payment);
        DeliveryThread t3=new DeliveryThread(delivery);
        t1.start();
        t1.join();
        t2.start();
        t2.join();
        if(!payment.equalsIgnoreCase("Failed")){
            t3.start();
            t3.join();
        }else{
            System.out.println("Delivery assignment should not proceed"
        }
    }
}
class OrderThread extends Thread{
    String order;
    OrderThread(String order){
        this.order=order;
    }
    public void run(){
        System.out.println("Order Processing: "+order);
    }
}
```

Input

```
Order=Burger; Payment=Card;
Delivery=Agent B
```

Output

```
Order Processing: Burger
Payment Processing: Card
Delivery Assignment: Agent B
```